

USDT Transaction Explorer

Software Requirements Specification

Table of Contents

1. INTRODUCTION	1
1.1 PURPOSE	1
1.2 SCOPE	1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	1
1.4 REFERENCES	1
1.5 OVERVIEW	1
2. GENERAL DESCRIPTION	2
2.1 PRODUCT PERSPECTIVE	2
2.2 PRODUCT FUNCTIONS	2
2.3 USER CHARACTERISTICS	2
2.4 GENERAL CONSTRAINTS	2
2.5 ASSUMPTIONS AND DEPENDENCIES	2
3. SPECIFIC REQUIREMENTS	3
3.1 EXTERNAL INTERFACE REQUIREMENTS	3
3.1.1 User Interfaces	3
3.1.2 Hardware Interfaces	3
3.1.3 Software Interfaces	3
3.1.4 Communications Interfaces	3
3.2 FUNCTIONAL REQUIREMENTS	3
3.2.1 Transaction Tree Generation	3
3.2.2 AI-Based Analysis of Wallet Transaction	3
3.3 NON-FUNCTIONAL REQUIREMENTS	4
3.3.1 Performance	4
3.3.2 Reliability	4
3.3.3 Availability	4
3.3.4 Security	4
3.3.5 Maintainability	4
3.3.6 Portability	4
3.4 DESIGN CONSTRAINTS	4

1. INTRODUCTION

This document outlines the software requirements for USDT Transaction Explorer, a system designed to analyze and visualize USDT (Tether) transaction data. The application assists businesses by retrieving wallet transaction data, visualizing transaction flows, and providing AI-based safety analysis regarding wallets before engaging in transactions.

The system is hosted on **AWS** and utilizes **LinodeGPU** for enhanced machine learning performance.

The project is hosted locally as of now.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the system requirements for the USDT Transaction Explorer, which helps users evaluate and analyze transaction history for USDT wallets. The system will provide transaction trees and safety assessments, helping businesses decide whether to engage with specific wallets.

1.2 Scope

The **USDT Transaction Explorer** includes:

- Retrieval of transaction data for ERC20 (Ethereum-based) and TRC20 (TRON-based) wallets via blockchain explorers.
- Visualization of wallet transaction data through transaction trees.
- AI-based analysis of wallet transaction history to provide safety assessments.
- Hosted on AWS using LinodeGPU to ensure high-performance processing during analysis.

Exclusions: The system will not perform financial audits or deep wallet verification. It will only assess transaction patterns and safety based on the wallet's historical behavior.

1.3 Definitions, Acronyms, and Abbreviations

- **USDT:** Tether, a stable coin pegged to the US Dollar.
- **ERC20:** A token standard on the Ethereum blockchain, typically used for USDT.
- **TRC20:** A token standard on the TRON blockchain for USDT.
- **AWS:** Amazon Web Services, the cloud platform for hosting applications.
- **LinodeGPU:** AWS GPU instances used for performing machine learning tasks related to transaction analysis.
- **API:** Application Programming Interface used for interacting with external services like Etherscan and Tronscan.
- **CI/CD Pipeline:** Continuous Integration/Continuous Deployment pipeline are a set of practices and tools designed to streamline software development cycles. Especially in terms of integrating and deploying code.
- **AI:** Artificial Intelligence used for analyzing wallet transaction data.

1.4 References

- **AWS Documentation:** [AWS Docs](#)
- **Etherscan API Documentation:** [Etherscan API](#)
- **Tronscan API Documentation:** [TRONSCAN API](#)

1.5 Overview

This document provides a comprehensive list of functional and non-functional requirements for the USDT Transaction Explorer. The system is designed to allow users to query wallet addresses, view transaction trees, and receive AI-based evaluations of wallet safety for business engagements. The AI model and analysis are powered by Google Generative AI, hosted on AWS.

2. GENERAL DESCRIPTION

2.1 Product Perspective

The USDT Transaction Explorer integrates with Etherscan and Tronscan APIs to fetch transaction data for USDT wallets. The system processes and visualizes this data as transaction trees. It then uses Google Generative AI to evaluate transaction patterns and provide safety recommendations for engaging with specific wallets.

2.2 Product Functions

- **Transaction Data Retrieval:** Fetch USDT transactions from Ethereum (ERC20) and TRON (TRC20) blockchains using Etherscan and Tronscan APIs.
- **Transaction Tree Generation:** Generate visual transaction trees to display wallet-to-wallet USDT transfers.
- **AI-Driven Analysis:** Analyze transaction history with AI and provide insights on whether it's safe to engage with the wallet.
- **Web Interface:** Users can interact with the system via a web interface to input wallet addresses and view transaction trees and safety analyses.

2.3 User Characteristics

The primary users of this system are businesses and analysts who need to assess the safety of transactions with USDT wallets. These users are expected to have a basic understanding of blockchain technology, cryptocurrencies, and wallet addresses.

2.4 General Constraints

- **Scalability:** The system must handle multiple concurrent users, especially during peak usage.
- **API Rate Limits:** Etherscan and Tronscan APIs impose rate limits, which need to be managed in the system.
- **Performance:** Transaction processing should be optimized, leveraging LinodeGPU for AI and data analysis.

2.5 Assumptions and Dependencies

- The system assumes stable access to Etherscan and Tronscan APIs.
- AWS infrastructure and LinodeGPU instances are used for hosting and processing.
- The AI model will provide consistent performance based on the historical data available.

3. SPECIFIC REQUIREMENTS

3.1 External Interface Requirements

3.1.1 User Interfaces

- A web interface where users can input wallet addresses (ERC20 or TRC20) and view transaction trees and AI safety assessments.

3.1.2 Hardware Interfaces

- **AWS EC2 Instances:** The system is hosted on AWS using LinodeGPU instances for machine learning tasks. The system's backend interacts with these instances for processing wallet data.
- **API:** Interaction with external APIs like Etherscan and Tronscan to retrieve wallet transaction data.

3.1.3 Software Interfaces

- **Etherscan API:** To fetch transaction details for ERC20 wallets.
- **Tronscan API:** To fetch transaction details for TRC20 wallets.
- **Google Generative AI API:** For analyzing transaction data and providing safety assessment.

3.1.4 Communications Interfaces

- **HTTPS:** Secure communication between the system and external APIs, and between the web interface and the backend.

3.2 Functional Requirements

3.2.1 Transaction Tree Generation

- **Introduction:** Retrieve transaction data for a given wallet address and create a transaction tree.
- **Input:** Wallet address (ERC20 or TRC20).
- **Processing:** System queries blockchain APIs (Etherscan or Tronscan) to fetch transaction data, which is processed and visualized as a tree structure.
- **Outputs:** Transaction tree visualizing wallet-to-wallet USDT transactions.
- **Error Handling:** If the wallet address is invalid or the API call fails, an error message is returned.

3.2.2 AI-Based Analysis of Wallet Transactions

- **Introduction:** Analyze wallet transactions using Generative AI to determine if it's safe to engage in business with the wallet.
- **Inputs:** Transaction tree data, generated in 3.2.1.
- **Processing:** Use Google Generative AI to analyze patterns and provide an opinion about the safety of transacting with the wallet.
- **Outputs:** A textual safety assessment provided by the AI model.
- **Error Handling:** If the AI analysis fails, the AI analyzes it and reports accordingly.

3.3 Non-Functional Requirements

3.3.1 Performance

- The system must be able to process up to 5 wallet queries per second, with efficient data retrieval and AI processing.

3.3.2 Reliability

- The system should maintain 99.9% uptime to ensure reliability for business operations.

3.3.3 Availability

- Hosted on AWS with auto-scaling support for dynamic workloads.

3.3.4 Security

- HTTPS communication is used for all external and internal connections to ensure data security.

3.3.5 Maintainability

- The system architecture should be modular and easy to maintain. All code is version-controlled and deployed using CI/CD pipelines.

3.3.6 Portability

- The system should be portable across different AWS regions to ensure geographic redundancy.

3.4 Design Constraints

- LinodeGPU instances are used for machine learning tasks to ensure fast processing of transaction data.
- The system is constrained by the rate limits of the Etherscan and Tronscan APIs, which may require rate-limiting mechanisms in the code otherwise thinks of the rapid requests as a DDOS attack and bans the IP.